

halfedge：内蕴 Delaunay 三角剖分设计文档

src/intrinsic.rs

halfedge 库

2026-07-05

目录

1	模块定位	1
1.1	与外蕴 Delaunay 的区别	2
1.2	API 总览	2
2	核心算法	2
2.1	内蕴 Delaunay 判据	2
2.2	翻转后新边长度	2
2.3	主算法流程	3
2.4	收敛性	3
3	内蕴边长系统	3
3.1	初始化	3
3.2	翻转后更新	4
4	复杂度	4
5	退化守卫	4
6	测试覆盖	5
7	设计权衡	5

1 模块定位

`intrinsic.rs` 实现内蕴 Delaunay 三角剖分 (Intrinsic Delaunay Triangulation, IDT)，基于 Fisher, Springborn, Schröder & Desbrun (2007) 的边翻转算法。IDT 保证所有内部边的余切拉普拉斯权重非负，从而保证离散 Laplace-Beltrami 算子的正定性。

1.1 与外蕴 Delaunay 的区别

	外蕴 Delaunay	内蕴 Delaunay
判据	外接圆包含对角顶点 (3D 空间)	$\alpha_C + \alpha_D > \pi$ (内蕴度量)
几何依据	嵌入空间中的圆	曲面上的边长度量
收敛性	不保证	保证 (Bobenko & Springborn 2007)
保持性	修改顶点位置	仅修改连接关系, 保持内蕴度量

1.2 API 总览

函数	功能	返回值
<code>intrinsic_delaunay</code>	IDT 主入口	<code>IntrinsicDelaunayStats</code>
<code>compute_intrinsic_lengths</code>	初始化内蕴边长	<code>HashMap<HalfEdgeId, f64></code>
<code>is_intrinsic_delaunay_edge</code>	单边 Delaunay 判据	<code>bool</code>
<code>intrinsic_cotan_weight</code>	内蕴余切权重	<code>f64</code>

2 核心算法

2.1 内蕴 Delaunay 判据

设内部边 $e = AB$, 两侧三角形为 ABC 和 ABD , C, D 为对角顶点。定义 α_C, α_D 为 C, D 处的内角。

Delaunay 条件:

$$\alpha_C + \alpha_D \leq \pi \iff \cot \alpha_C + \cot \alpha_D \geq 0$$

当 $\cot \alpha_C + \cot \alpha_D < 0$ 时, 翻转边 $AB \rightarrow CD$ 能改善 Delaunay 性质。

用边长表达:

$$\cot \alpha_C = \frac{l_{CA}^2 + l_{BC}^2 - l_{AB}^2}{4 \cdot \text{Area}(ABC)}, \quad \cot \alpha_D = \frac{l_{AD}^2 + l_{BD}^2 - l_{AB}^2}{4 \cdot \text{Area}(ABD)}$$

其中三角形面积由 Heron 公式:

$$\text{Area}(a, b, c) = \sqrt{s(s-a)(s-b)(s-c)}, \quad s = \frac{a+b+c}{2}$$

2.2 翻转后新边长度

边 $e = AB$ 被翻转为 CD 后, 新边的内蕴长度 l_{CD} 由余弦定理计算:

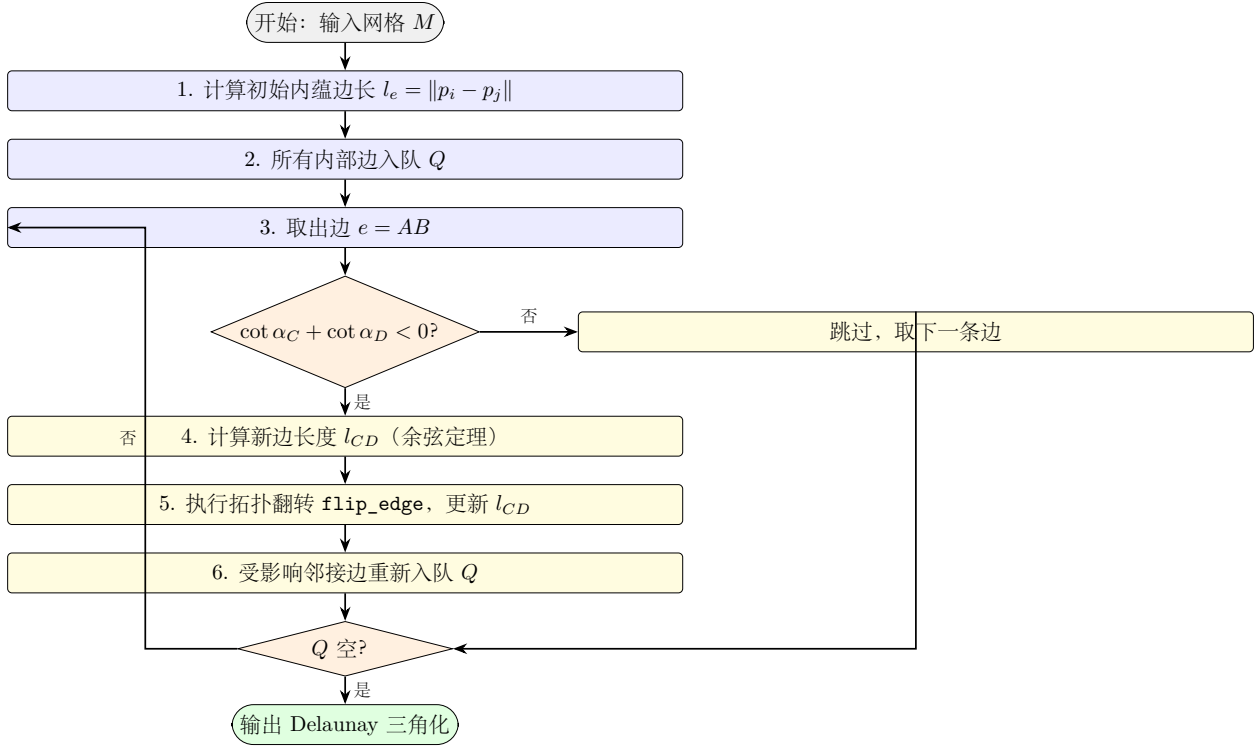
$$l_{CD}^2 = l_{CA}^2 + l_{AD}^2 - 2 \cdot l_{CA} \cdot l_{AD} \cdot \cos(\angle A_{ABC} + \angle A_{ABD})$$

其中:

$$\angle A_{ABC} = \arccos \frac{l_{AB}^2 + l_{CA}^2 - l_{BC}^2}{2 \cdot l_{AB} \cdot l_{CA}}, \quad \angle A_{ABD} = \arccos \frac{l_{AB}^2 + l_{AD}^2 - l_{BD}^2}{2 \cdot l_{AB} \cdot l_{AD}}$$

关键性质: 翻转不改变已有边的内蕴长度, 只有新边 CD 的长度需要计算。这是内蕴三角化的核心——度量只取决于曲面本身, 不取决于三角化方式。

2.3 主算法流程



2.4 收敛性

Bobenko & Springborn (2007) 证明: 定义 Dirichlet 能量

$$E_D(\mathcal{T}) = \frac{1}{4} \sum_{e \in E(\mathcal{T})} (\cot \alpha_C(e) + \cot \alpha_D(e)) \cdot l_e^2$$

每次合法翻转使 E_D 严格递减, 而 $E_D \geq 0$ 且可能的三角化数目有限, 故算法在有限步内终止。设安全上限为 $\text{max_iterations} = 10 \cdot |E|$ 。

3 内蕴边长系统

3.1 初始化

内蕴边长初始化为 3D 欧氏距离:

```

1 pub fn compute_intrinsic_lengths(mesh: &MeshStorage)
2   -> HashMap<HalfEdgeId, f64>
3 {
4   let mut lengths = HashMap::new();
5   for he in mesh.halfedge_ids() {
6     if let Some(l) = edge_length(mesh, he) {
7       lengths.insert(he, l);
8     }
9   }
10  lengths
11 }
  
```

twin 对的两个半边具有相同的长度。

3.2 翻转后更新

翻转 $AB \rightarrow CD$ 后, flip_edge 保留半边 ID, 仅修改连接关系。更新 intrinsic_lengths:

```
1 intrinsic_lengths.insert(he, l_cd);    // he: D->C
2 intrinsic_lengths.insert(twin, l_cd);   // twin: C->D
```

已有边 (AC, CB, BD, DA) 的内蕴长度不变。

4 复杂度

阶段	时间复杂度	备注
初始边长计算	$O(E)$	每条半边一次 edge_length
队列初始化	$O(E)$	遍历所有无向边
单次翻转	$O(1)$	余弦定理 + flip_edge
总翻转次数	$O(E)$	实际经验值
受影响边入队	$O(1)$ per flip	最多 8 条邻接边
总计	$O(E)$ (实际)	安全上限 $10E$

5 退化守卫

- 退化三角形 (面积 $< 10^{-14}$) 视为满足 Delaunay 条件, 不翻转;
- acos 输入用 clamp(-1.0, 1.0) 保护, 防止 NaN;
- l_cd_sq.max(0.0).sqrt() 防止负数开方;
- 最大迭代次数 $10 \cdot |E|$ 防止意外无限循环;
- 翻转前检查 contains_halfedge 和 is_interior_edge, 防止操作已删除或边界半边;
- flip_edge 失败时跳过 (DegenerateTriangle 等)。

6 测试覆盖

测试名	验证点
<code>intrinsic_delaunay_grid</code>	grid 翻转后所有边满足 Delaunay
<code>intrinsic_delaunay_icosphere_no_flip</code>	icosphere IDT 正确
<code>intrinsic_delaunay_flat_quad</code>	扁平四边形翻转后满足 Delaunay
<code>is_delaunay_equilateral</code>	等边三角形满足 Delaunay
<code>triangle_area_heron</code>	Heron 公式正确性
<code>flipped_length_diamond</code>	菱形翻转后新边长度 = 2
<code>intrinsic_lengths_match_euclidean</code>	初始长度与欧氏距离一致
<code>cotan_weight_nonnegative_after_delaunay</code>	翻转后余切权重 ≥ 0
<code>intrinsic_delaunay_empty</code>	空网格不 panic
<code>intrinsic_delaunay_single_triangle</code>	单三角形无内部边不翻转

全模块共 10 个单元测试；项目整体 `cargo test` 共 460 个用例。

7 设计权衡

- **独立 HashMap vs 属性系统**：选择 `HashMap<HalfEdgeId, f64>` 存储内蕴边长，因为 IDT 算法需要频繁读写边长，属性系统的间接开销较大。缺点是需手动维护与网格拓扑的同步。
- **余切判据 vs 角度判据**：使用 $\cot \alpha_C + \cot \alpha_D < 0$ 而非 $\alpha_C + \alpha_D > \pi$ ，因为余切可直接从边长计算，无需调用 `acos`（数值更稳定）。
- **VecDeque + HashSet 队列**：使用 BFS 式队列而非优先队列，因为 IDT 不需要优先级（任何不满足条件的边都可以翻转，顺序不影响结果）。HashSet 避免同一条边重复入队。
- **与 `remesh::flip_for_valence` 的区别**：remesh 翻转以优化顶点度数（外蕴目标），IDT 翻转以优化内蕴几何性质。两者不应混用。
- **安全上限 10E**：理论上翻转次数 $O(E)$ ，但设 10E 安全上限防止极端情况。